Guía de estudio — TP3, Ejercicio 5

Planificación FIFO. Sistemas de Tiempo Real

1. Los números de memoria

El cambio: dos valores. En la biblioteca, `configUSE_TIME_SLICING` de 1 a 0, que es lo que pide el enunciado. En el firmware, `USAR_ROUND_ROBIN` de 1 a 0, con lo que desaparecen los `taskYIELD()` y la ráfaga se ejecuta completa. El proyecto no compila si quedan desacoplados.

Dato incómodo que conviene decir antes de que lo pregunten: en el Ejercicio 4 se hizo la prueba inversa, dejar `configUSE_TIME_SLICING` en 1 y quitar solo el `taskYIELD()`, y el resultado fue idéntico al de acá. O sea que en este simulador el parámetro de la biblioteca no tiene efecto observable; lo que produce el comportamiento FIFO es la ausencia de `taskYIELD()`.

Ráfagas atómicas: A, B y C miden **128 ms**; D mide **64 ms**. Se verifica restando las dos columnas del cronograma.

Orden de atención: $B \to C \to D \to A$. B toma el procesador a los 15 ms, C a los 143, D a los 271, A completa a los 705.

	Respuesta (instante crítico)		Plazos perdidos		
Tarea	Round Robin	FIFO	Round Robin	FIFO	T
A	507 ms	705 ms	0	0	768
B	523 ms	**143 ms**	0	0	1536
C	529 ms	**271 ms**	1 / hiperp.	**0**	384
D	257 ms	335 ms	1 / hiperp.	**3 / hiperp.**	192

Las cuatro tareas piden $3 \times 128 + 64 = \mathbf{448}$ ms de cómputo y el procesador los sirve igual de rápido en las dos políticas, sin quedar ocioso mientras haya ráfagas pendientes. Lo que cambia es el orden, y por lo tanto a quién se le cobra la espera.

2. Preguntas probables y respuesta corta

- **¿Qué cambiaste exactamente?** Dos valores: `configUSE_TIME_SLICING` a 0 en `FreeRTOSConfig.h`, que es lo que pide el enunciado, y `USAR_ROUND_ROBIN` a 0 en el firmware, con lo que la tarea deja de ceder el procesador al cerrar cada cuanto. Ningún otro archivo de la biblioteca se modifica.
- **¿Entonces esto es FIFO de verdad?** Lo que es FIFO es la *ejecución*: la ráfaga no se interrumpe. La lista de listos de FreeRTOS, en cambio, no es una cola por orden de llegada. Conviene no confundir las dos cosas.
- **¿Por qué el orden no es el de llegada?** `prvAddTaskToReadyList` llama a `vListInsertEnd`, que inserta *delante del índice actual* de una lista circular, y `listGET_OWNER_OF_NEXT_ENTRY` *avanza ese índice* antes de devolver el dueño. El índice persiste entre selecciones, así que el recorrido arranca desde donde había quedado.
- **¿Sigue siendo expropiativo?** Sí, entre prioridades distintas: la tarea de fondo es desalojada en cuanto se libera cualquier periódica. Lo que desaparece es el reparto por cuantos entre tareas de igual prioridad.
- **¿En qué situación podría producirse igualmente una expropiación?** Es la pregunta del inciso y el cronograma la contesta en una sola tarea: A nunca ejecuta sus 128 ms de corrido. Su primera ráfaga sale en `1-15`, `335-396` y `652-705`, que suman 128. A C le pasa de vez en cuando (`1152-1157` más `1221-1344`); B y D nunca se parten. La causa son los bordes donde el despachador reanuda el planificador: `xTaskResumeAll()` procesa los ticks que quedaron pendientes y, si alguno se procesa y hay otra tarea lista, cede el procesador antes de devolver el control. Por eso los cortes caen en múltiplos de los 16 ms con que se trocea la ráfaga, y por eso son esporádicos.
- **¿Por qué B mejora tanto, de 523 a 143 ms?** Porque es la primera atendida y se lleva el procesador entero: tarda exactamente su cómputo. Bajo Round Robin avanzaba a un cuarto de velocidad.
- **¿Por qué A empeora?** Queda última en el recorrido de la lista, y mientras espera la pasan por encima las tareas que se reactivan. Pasa de 507 a 705 ms.
- **¿Por qué D empeora, si su ráfaga es la más corta?** Porque con ráfagas atómicas puede quedar bloqueada hasta 128 ms detrás de una tarea cuyo plazo no corre ningún riesgo. D tiene que terminar cada 192 ms; una espera de 128 se le come dos tercios del margen. Pasa de 1,13 a 2,99 plazos perdidos por hiperperíodo, o sea el 37,3 % de sus activaciones: 103 sobre 276.
- **¿Cuál de las dos políticas es mejor?** Ninguna domina a la otra. Round Robin reparte el perjuicio entre C y D; FIFO lo concentra en D. Preguntarse cuál es mejor es la pregunta equivocada: las dos ignoran el mismo dato.
- **¿Qué información ignoran los dos planificadores?** El período y el plazo de cada tarea. Round Robin solo conoce el cuanto, FIFO solo el orden de la lista. Ninguno recibe el dato de que D debe terminar cada 192 ms

mientras B dispone de 1536.

- **¿Y entonces qué habría que usar?** Prioridades asignadas por período (Rate Monotonic) o por plazo más próximo (Earliest Deadline First). Ver el punto 3.

3. El cierre fuerte: con Rate Monotonic se cumplen todos los plazos

Conviene tenerlo calculado, porque conecta con el Ejercicio 1(a) y es la conclusión natural del trabajo.

Con prioridades por período (D la más alta, después C, A y B), el análisis de tiempo de respuesta $R_i = C_i + \sum_{j \in hp(i)} \lceil R_i/T_j \rceil \, C_j$ da, en unidades del enunciado:

Tarea	R (unidades)	T	¿Cumple?
D	1	3	sí
C	3	6	sí
A	6	12	sí
B	12	24	sí

Las cuatro cumplen. Y el detalle que vale la pena señalar: la cota de Liu y Layland para $n = 4$ vale $4(2^{1/4} - 1) = 0{,}757$, y la utilización del conjunto es $U = 0{,}9167$, o sea que *la cota no se cumple*. Pero la cota es suficiente y no necesaria: el análisis exacto de tiempo de respuesta muestra que el conjunto sí es planificable. Es exactamente lo que pedía discutir el Ejercicio 1(a).

Dicho de otro modo: sobre el mismo hardware y con el mismo código de las tareas, lo único que hay que cambiar para que no se pierda ningún plazo es **qué sabe el planificador**.

4. Las tres cosas que no hay que decir mal

- FIFO **no** significa que se atienda en orden de llegada. Significa que la ráfaga no se interrumpe. El orden observado es $B \to C \to D \to A$ y las tareas se crean A, B, C, D.
- FIFO **no** deja de ser expropiativo. Entre prioridades distintas la expropiación sigue intacta, y entre iguales todavía aparece en los bordes donde se reanuda el planificador: A queda partida en tres tramos en cada activación.
- FIFO **no** es mejor que Round Robin. Es distinto: mejora a las atendidas primero y empeora a las últimas.

5. Cómo mostrarlo, en un minuto

1. Mostrar los dos cronogramas uno al lado del otro. Round Robin: tramos de 16 ms, más de treinta cambios de contexto. FIFO: seis tramos en el mismo intervalo.
2. Restar las dos columnas de una línea de FIFO: `B 15 143` son 128 ms, el cómputo entero. Eso es la atomicidad.
3. Señalar los tres tramos de A: `1-15`, `335-396` y `652-705`, que suman 128. Ahí está la expropiación que sobrevive, en los bordes donde se reanuda el planificador.
4. Cerrar con la tabla comparada: B y C mejoran, A y D empeoran, el trabajo total no cambia. Y con el cálculo de Rate Monotonic, que cumple los cuatro plazos.